

# **\*\*AKS Course — Part 2 (Markdown■Styled PDF)\*\***

## **\*\*Sections 1–3\*\***

### 1. Introduction to Kubernetes (K8s)

What is Kubernetes?

Kubernetes is an open-source container orchestrator.

The word comes from the Greek word “Helmsman” meaning one who steers a ship (like a container ship).  
It automates:

Deployment of containers

Scaling of applications

High availability

Load balancing

Resource allocation

Automated rollouts & rollbacks

What does “K8s” mean?

It is shorthand for Kubernetes:

“K” + 8 letters + “s”

Why do we need Kubernetes?

Running one or two containers manually is easy.

Running hundreds or thousands, across multiple servers, with:

Auto-scaling

Self-healing

Zero-downtime deployments

...requires a system like Kubernetes.

### 2. Kubernetes Cluster Basics

A Kubernetes cluster consists of:

a) Control Plane (Master Nodes)

This is the “brain” of the cluster.

It manages:

Desired state

Scheduling pods

Monitoring nodes

Kubernetes objects

Communications

Cluster-wide decisions

What is a Node?

A node is a VM or physical machine inside the cluster.

Two types:

Master node / Control plane node → manages cluster

Worker node / Data plane node → runs your applications (pods)

What is a Pod?

A pod is the smallest unit in Kubernetes.

You cannot run a container directly – it must run inside a pod.

A pod:

Can contain one or more containers

Shares:

Same network namespace

Same IP address

Same storage

Multiple containers inside a pod communicate via localhost.

What is a Deployment?

A deployment manages the lifecycle of pods.  
With a deployment you define:

- How many replicas you want
- Update strategy
- Rollbacks

Deployments automatically create a ReplicaSet, which ensures the correct number of pods always exist.  
What is a Service?  
Kubernetes service = a stable way to access pods.  
It:

- Has its own IP (from Service CIDR)
- Load-balances traffic
- Provides stable networking even if pods restart

### 3. Kubernetes Architecture (Deep Dive)

#### 3.1 Control Plane Components

Let's break every component down with clear definitions.

##### ■ a) Etcd – "The Database of Kubernetes"

Etcd is a distributed key-value store used to store all cluster state, including:

- Nodes
- Pods
- Services
- Secrets
- ConfigMaps
- Cluster configuration

Whenever you run:  
kubectl get <anything>

→ You are reading from Etcd (via the API server).  
You never interact with Etcd directly.  
Only the API server talks to Etcd.

##### ■ b) API Server – "The Front Door of Kubernetes"

The API server:

- Exposes the Kubernetes API over HTTPS
- Validates requests
- Authenticates users
- Authorizes actions
- Updates Etcd
- Communicates with all control plane and worker node components

Every request from:

- kubectl CLI
- Controllers
- Schedulers
- Kubelet

...passes through the API server.

##### ■ c) Controller Manager – "Maintains Desired State"

A controller continuously watches the cluster and corrects deviations.  
The Controller Manager bundles many controllers:

- Node Controller → monitors node health
- Replication Controller → ensures correct pod count
- Deployment Controller → manages deployments
- And many others

Simple example:  
If you have a Deployment with 3 pods and one pod crashes:

Controller detects difference  
Creates another pod  
Ensures state always matches "desired state"

#### ■ d) Scheduler – "Decides Where Pods Run" Scheduler:

Watches for unscheduled pods  
Selects the best node  
Considers:

Available CPU/RAM  
Pod resource requests  
Node Taints  
Pod Tolerations  
Affinity rules  
Storage / topology

Example:  
If a pod needs 3 GB RAM, and nodes have:

Node A: 2 GB free  
Node B: 4 GB free  
Node C: 6 GB free

Scheduler picks Node C (most space left).

#### ■ e) Cloud Controller Manager (Only in Cloud) Used when Kubernetes runs in a cloud platform like Azure. It integrates with cloud provider APIs and manages cloud resources:

Load balancers  
Public IPs  
Storage disks  
VM lifecycle

This becomes very important in AKS.

### 3.2 Worker Node Components Worker nodes run your applications.

#### ■ a) Kubelet – "The Node Agent" Kubelet:

Runs on every worker node  
Talks to API server  
Ensures containers defined in pods are actually running  
Reports status back to control plane  
Communicates with container runtime

It keeps actual state close to desired state.

#### ■ b) Container Runtime (containerd in AKS) What is a container runtime? It is the software that actually runs containers. Examples:

Docker  
containerd (used by AKS)  
CRI-O

Container runtime:

Pulls images  
Starts containers

Stops containers  
Manages storage & networking

### ■ c) Kube-proxy – “Network Traffic Manager”

Kube-proxy configures:

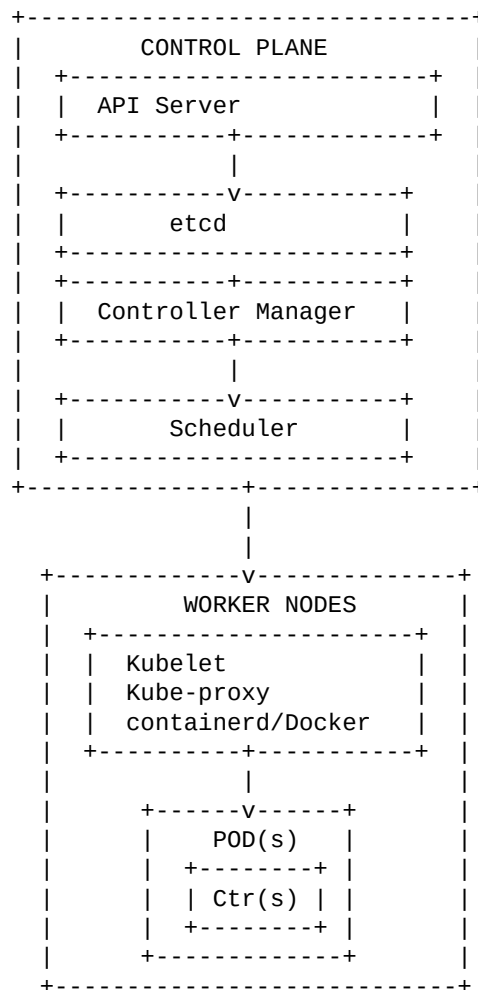
IPTables / IPVS rules  
Load-balancing  
Pod-to-pod traffic routing  
Service routing

If you create a Kubernetes Service:

Kube-proxy ensures traffic gets forwarded

Works on every node

[Diagram] Kubernetes High-Level Architecture



## **\*\*Sections 4–6\*\***

### 4. What Happens When a Developer Creates a Pod? (Step-by-Step Explanation)

Let's walk through the entire flow.

Step 1: Developer runs:

```
kubectl create -f pod.yaml
```

Step 2: kubectl converts YAML → JSON

Kubernetes internally uses JSON.

Step 3: API Server receives request

Validates YAML

Checks authentication

Checks authorization (RBAC)

Writes pod definition to Etcd

Step 4: Scheduler gets notified

Scheduler:

Selects a suitable node

Updates API server with its decision

API server updates Etcd

Step 5: Kubelet on selected node is notified

API server tells kubelet to create pod

Kubelet calls container runtime

Container runtime pulls image & runs container

Step 6: Kubelet reports status to API Server

Status → API server → Etcd → reflected in `kubectl get pods`

### 5. What Happens When We Use a Deployment Instead of a Pod?

With a Deployment:

Controller manager creates a ReplicaSet

ReplicaSet:

Creates pods

Keeps number of replicas correct

Recreates missing pods

This ensures high availability.

### 6. How Users Access Applications in the Cluster

Apps run in pods, but pods:

Restart

Move between nodes

Have dynamic IPs

Thus, users cannot access pods directly.

Solution → Kubernetes Service (LoadBalancer)

Process:

You create a Service of type LoadBalancer

API server processes it

Cloud controller manager creates:

Azure Load Balancer

Public IP

Kube-proxy configures routing on nodes

External users access app via the public IP

[Diagram] Pod Creation Flow

kubectl apply --> API Server --> etcd (store desired state)

|  
v  
Scheduler (pick node)  
|  
v  
API Server --> etcd (bind)

|  
v  
Kubelet (on chosen node)

|  
v  
Container Runtime (pull image, run)

|  
v  
Kubelet --> API Server --> etcd (status)

[Diagram] Service (LoadBalancer) Path

Client --> Public IP (Azure LB) --> Node (kube-proxy rules) --> Pod IP

## **\*\*Sections 7–10\*\***

### 7. Self-Managed Kubernetes vs Cloud-Managed AKS

You thoroughly compared both.

#### ✓■ Creation Complexity

■ Self-managed → install everything manually

✓■ Cloud-managed → cluster in minutes

#### ✓■ Flexibility

✓■ Self-managed → full customization

■ Cloud-managed → limited by provider rules

#### ✓■ Management Overhead

■ Self-managed → you manage everything

✓■ AKS → Azure manages control plane & components

#### ✓■ Required Skill

■ Self-managed → deep Kubernetes knowledge

✓■ AKS → easier, less ops overhead

#### ✓■ Security

AKS provides:

Azure AD integration

RBAC

Automatic patching

SOC/ISO/HIPAA compliance

#### ✓■ Ecosystem Integration

In AKS, you get:

Azure load balancers

Azure firewalls

Azure Monitor

Defender for Cloud

ACR integration

#### ✓■ Cost

AKS control plane is free

Only pay for worker nodes + associated services

#### ✓■ Support

Cloud → Azure support available

Self-managed → community or your own team

### 8. AKS Features & Benefits (Your Notes Summarized)

AKS provides:

Fast cluster creation

Managed control plane

Auto/manual scaling

Auto/manual upgrades

High security

Easy monitoring

Excellent Azure ecosystem integration

Responsibility Split:

Azure manages → control plane

You manage → worker nodes

Both must follow AKS support policies

Definition

Azure Kubernetes Service (AKS) is a fully managed Kubernetes service provided by Microsoft Azure.

“Managed” means Azure takes responsibility for installing, configuring, upgrading, and maintaining the

What Azure Manages (Control Plane)

Azure automatically manages:

API Server

Scheduler

Controller Manager

etcd (cluster state DB)

Cloud Controller Manager

Certificates & security patches

Health monitoring

Control-plane upgrades

High availability of master components

Important: The control plane in AKS is free.

What YOU Manage (Worker Nodes / Node Pools)

Your part of responsibility includes:

Worker nodes (VMs running your pods)

Node OS updates / upgrades

Autoscaling behavior

Networking configuration

Add-ons (Ingress, DNS, monitoring)

Application deployments

Security policies (RBAC, pod security, network security)

Azure applies a shared responsibility model:

Azure manages the “brains” of Kubernetes

You manage the “muscle” (nodes + applications)

## 10. Azure Free Account + Pricing Model – Explained Clearly

Azure Free Account

Azure offers:

\$200 free credit for 30 days

12 months of free services

Some services are always free

For learning AKS, the free account is perfect because:

AKS control plane = always free

You only pay for worker nodes

You only pay for any storage, ingress, load balancers, public IPs, etc.

Azure Pricing Models

1. Pay-As-You-Go (Default)

No commitment

Highest hourly rate

Best for learning and development

2. Reserved Instances (1- or 3-year commitments)

You reserve VM sizes for long term

Up to 65% cheaper than pay-as-you-go

Best for production systems

3. Spot Instances

Spot = unused Azure capacity sold very cheaply

Up to 90% cost savings

Can be evicted at any time



Useful for:

Batch jobs  
Non-production workloads  
Test environments

Spot nodes are NOT recommended for production because of interruptions.  
Region-Based Pricing  
Different regions (West Europe vs East US) have different price points.  
Always check Azure's pricing calculator before deploying.

## **\*\*Sections 11–12\*\***

### 11. AKS Cost Breakdown – What You Actually Pay For

AKS pricing is unique because:

AKS Control Plane is FREE

You pay \$0 for:

API Server

etcd

control plane VMs

load-balancing of control plane

control plane auto-upgrades

Unless you choose Standard Tier (Uptime SLA).

What Costs Money in AKS?

#### ✓■ 1. Worker Nodes (VMs)

These are the actual compute nodes running your pods.

#### ✓■ 2. Managed Disks (Storage)

For:

Node OS disks

Persistent volume claims

ACR storage (if using Azure Container Registry)

#### ✓■ 3. Networking Resources

Load balancers

Public IP addresses

Bandwidth

Virtual network costs (minimal)

#### ✓■ 4. Monitoring & Logging

Using Azure Monitor / Log Analytics Workspace.

#### ✓■ 5. Optional Add■Ons

Such as:

Application Gateway Ingress Controller

Azure Firewall

Defender for Cloud

### 12. Cost Optimization Strategies for AKS (Explained Properly)

These are officially recommended ways to reduce AKS costs.

#### 12.1 Choose the Right VM Size

Node pools use VMs.

You choose size based on:

CPU

RAM

Disk

Network throughput

Important details:

System node pools must use  $\geq 2$  vCPUs

System node pools should not use B■series (burstable) VMs

Because system workloads are continuous, not burstable

User node pools can use cheaper burstable VMs

#### 12.2 Use Cluster Autoscaler

Cluster Autoscaler automatically:

Adds nodes when pods cannot be scheduled

Removes empty nodes to save money

Located under:

Node Pools → Scale → Autoscale

Autoscaling is essential for:

Predictable cost

Efficient resource usage

Handling sudden traffic spikes

### 12.3 Stop & Start Your AKS Cluster

If you're learning or using non-production environments:

You can stop the cluster when not needed

Azure preserves the cluster state for 12 hours

Saves 100% of node compute cost

Find this in AKS Overview:

Stop (to save money)

Start (when you resume work)

Node pools with mode System cannot be stopped individually  
–but user node pools can be.

### 12.4 Use Spot Node Pools (Cheap but Interruptible)

Spot VMs = up to 90% cheaper

Use spot node pools for:

Dev environments

Batch jobs

Non-critical workloads

CI/CD job runners

Do NOT use spot for:

System node pools

Production-critical workloads

### 12.5 Use Virtual Nodes (ACI Integration)

Virtual nodes run pods in:

Azure Container Instances (ACI)

Advantages:

Pay per second

Instant scale-out

No VM provisioning

Perfect for unpredictable traffic bursts

Enable via:

Virtual Nodes Add-on during cluster creation

### 12.6 Use Standard Tier Only for Production

AKS Pricing tiers:

Free Tier

Control plane free

No uptime SLA

Perfect for testing, learning, small workloads

Standard Tier (Paid)

\$0.10 per cluster per hour  
99.95% cluster uptime (when using Availability Zones)  
Recommended for production

#### 12.7 Region Selection

Choose the cheapest region close to your users.  
Prices vary A LOT between regions.

#### 12.8 Monitoring Cost Control

Log Analytics Workspace can become expensive.  
Ways to save:

Reduce retention  
Reduce log collection  
Exclude noisy namespaces (like kube-system)